SINGAPORE UNIVERSITY OF TECHNOLOGY AND DESIGN

50.039 Theory and Practice of Deep Learning

Project Report

Credit Card Fraud Detection

Anomaly Detection on Highly Imbalanced Dataset

Group 3

Name	Student ID
Pagdilao Geoffrey Cyd Caraig	1008183
Lindero Dianthe Marithe Lumagui	1007213
Karen Neo	1007005

Table of Contents

1. Introduction....4
2. Problem Definition....4
3. Dataset Description and Preprocessing....5
3.1 Dataset Overview.... 5
3.2 Exploratory Data Analysis....6
3.3 Data Splitting.... 8
3.4 Data Preprocessing.... 8
3.5 Class Imbalance Handling.... 9
4. Methodology.... 9
4.1 Models Implemented.... 9
4.1.1 Baseline Feedforward Neural Network (BNN).... 9
4.1.2 Weighted Feedforward Neural Network (WNN)....10
4.1.3 XGBoost....10
4.1.4 Autoencoder....11
4.2 Overall Training Setup.... 12
4.3 Hyperparameter Tuning.... 13
5. Results....13
5.1 Evaluation Metrics.... 13
5.2 Training Curves & Evaluation Metrics....13
5.3 Final Test Performance....18
5.4 Threshold Analysis.... 19
6. Model Limitations.... 20
6.1 Incorrect Model Implementation.... 20
6.2 Model-Specific Failure Patterns and Malfunction Examples....21
6.3 Feature Limitations.... 21
6.4 Model Architecture Limitations....21
6.5 Threshold Sensitivity....22
7. Potential Improvements....22
8. Discussion....22
9. Reproducibility / How to Run....23
9.1 Environment....23
9.2 Dataset.... 23
9.3 GitHub Repository.... 23
9.3.1 How to navigate.... 24
9.4 Steps to Run.... 24
10. Team Contributions.... 25
11. Conclusion.... 25

Abstract

Credit card fraud detection is a challenging problem due to the highly imbalanced nature of transaction data, where fraudulent cases are rare but critical to identify. This project investigates multiple machine learning approaches for detecting fraudulent transactions using a real-world tabular dataset.

Four models were developed and evaluated: a baseline feedforward neural network (BNN), a weighted neural network (WNN) with class imbalance handling, a tree-based XGBoost model, and an unsupervised autoencoder for anomaly detection. Performance was assessed using metrics suitable for imbalanced classification, including precision, recall, F1-score, ROC-AUC, and PR-AUC.

The results show that the baseline model achieves high accuracy but performs poorly in detecting fraud. Introducing class weighting improves recall and F1-score, while XGBoost provides the best overall balance between precision and recall. The autoencoder offers an alternative anomaly detection approach but demonstrates less consistent performance.

Overall, the study highlights the importance of handling class imbalance and selecting appropriate evaluation metrics. It also shows that tree-based models can be highly effective for structured tabular data, providing practical insights for real-world fraud detection systems.

1. Introduction

Credit card fraud detection refers to the task of identifying suspicious transactions that may have been conducted without the cardholder's consent. In practical terms, the objective is to detect fraudulent activity as early as possible, ideally before a customer is charged for an unauthorized purchase.

In real-world financial systems, this problem is highly dangerous. Each undetected fraudulent transaction can result in direct financial losses, additional investigation costs, and, more broadly, diminished trust in the financial institution or payment platform. Consequently, incorrectly flagging legitimate transactions may inconvenience customers by blocking valid purchases. As such, fraud detection extends beyond a purely technical challenge and has significant implications for both business risk management and customer experience.

Furthermore, credit card fraud detection is more appropriately framed as an anomaly detection problem rather than a standard balanced classification task. Fraudulent transactions typically constitute a very small proportion of all transactions, resulting in highly imbalanced datasets. Under such conditions, models may achieve high overall accuracy while failing to detect fraudulent cases entirely. As such, evaluation should prioritize metrics that reflect performance on the minority class, particularly the model's ability to detect fraud and minimize missed cases, rather than relying solely on accuracy.

Throughout this project, the primary objective was to develop and compare multiple machine learning approaches for credit card fraud detection, and to identify the most practical model for real-world deployment. This includes achieving a careful balance between effective fraud detection and the minimization of harmful errors, with particular emphasis on reducing false negatives. Given the rarity of fraudulent transactions, the central challenge was not in maximizing overall accuracy, but in accurately identifying the minority fraud class while maintaining a low rate of missed detections.

2. Problem Definition

The project is primarily a binary classification task (fraud vs non-fraud), but it behaves like anomaly detection as fraudulent transactions are extremely rare.

This dataset presents transactions that occurred in two days, where we have 492 frauds out of 284,807 transactions. The dataset is highly unbalanced, the positive class (frauds) account for 0.172% of all transactions.

Taken from the kaggle dataset description

A key challenge in this task is the severe class imbalance present in the dataset: **only 492 out of 284,807** transactions are fraudulent, accounting for approximately **0.172%** of all observations. Under such conditions, accuracy becomes a misleading performance metric, as a model can classify nearly all transactions as non-fraudulent and still achieve a deceptively high accuracy score.

In this context, model performance should instead be defined by its ability to reliably detect fraudulent transactions while minimizing missed cases (false negatives) and maintaining false positives at a manageable level. Accordingly, evaluation places greater emphasis on metrics that capture performance on the minority class, including the Precision-Recall Area Under the Curve **(PR-AUC)**, **recall**, **precision**, and the **F1 score**, rather than relying on accuracy alone.

3. Dataset Description and Preprocessing

3.1 Dataset Overview

We will be using the [Credit Card Fraud Detection]() from Kaggle.

The dataset contains anonymized credit card transactions made by European cardholders.
It contains:

- 289,807 transactions (total)
- 493 fraudulent transactions (~0.17%)

This makes the dataset highly imbalanced.

Inputs	1. Time
	  ○ Number of seconds elapsed between this transaction and the first transaction in the dataset
	
	2. Amount
	  ○ Transaction amount
	
	3. V1 to V28 Values
	  ○ Result of a PCA Dimensionality reduction (for confidentiality) to p

Outputs	● Class 0 → Non-fraudulent transaction ● Class 1 → Fraudulent transaction

3.2 Exploratory Data Analysis

To understand the dataset before model training, we performed exploratory analysis on the transaction records and class labels.

Dataset preview and class summary table.

	Time	V1	V2	V3	V4	V5	V6	V7	V8	V9	...	V21	V22	V23	V24	V25	V26	V27	V28	Amount	Class
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	0.098698	0.363787	...	-0.018307	0.277838	-0.110474	0.066928	0.128539	-0.189115	0.133558	-0.021053	149.62	0
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803	0.085102	-0.255425	...	-0.225775	-0.638672	0.101288	-0.339846	0.167170	0.125895	-0.008983	0.014724	2.69	0
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461	0.247676	-1.514654	...	0.247998	0.771679	0.909412	-0.689281	-0.327642	-0.139097	-0.055353	-0.059752	378.66	0
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609	0.377436	-1.387024	...	-0.108300	0.005274	-0.190321	-1.175575	0.647376	-0.221929	0.062723	0.061458	123.50	0
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941	-0.270533	0.817739	...	-0.009431	0.798278	-0.137458	0.141267	-0.206010	0.502292	0.219422	0.215153	69.99	0

5 rows × 31 columns

A sample preview of the table confirms that all variables are numeric, with features V1 to V28 that's transformed, including Time, Amount, and Class.

	Class	Count	Percentage
0	Non-Fraud (0)	284315	99.8273
1	Fraud (1)	492	0.1727

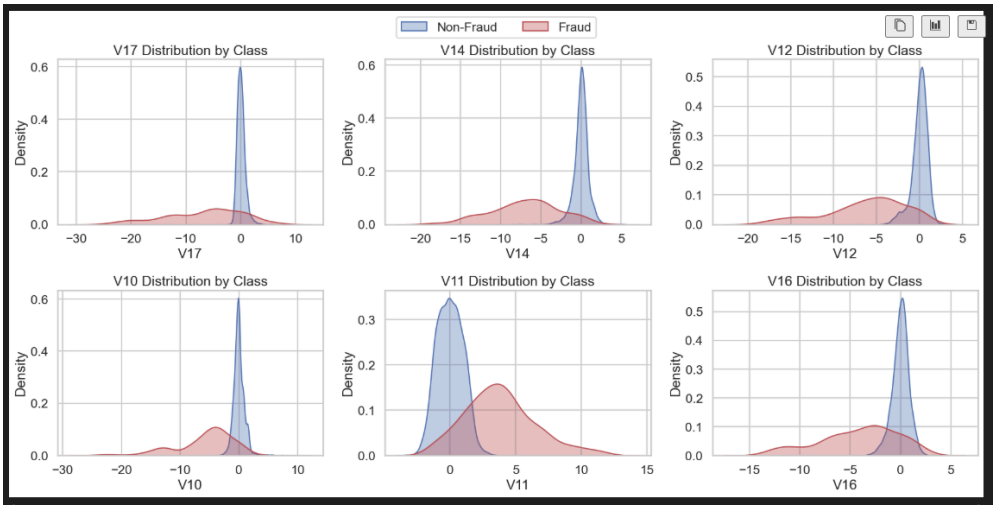
The class summary shows an extreme imbalance: with 284,315 non-fraud transactions and 492 fraud transactions, meaning fraud is only 0.1727% of the dataset confirming that the problem is highly skewed.

```
Top features selected by mutual information:
```

	Feature	MutualInformation
16	V17	0.008258
13	V14	0.008136
11	V12	0.007601
9	V10	0.007530
10	V11	0.006831
15	V16	0.006144
3	V4	0.004976
2	V3	0.004952
17	V18	0.004317
8	V9	0.004277

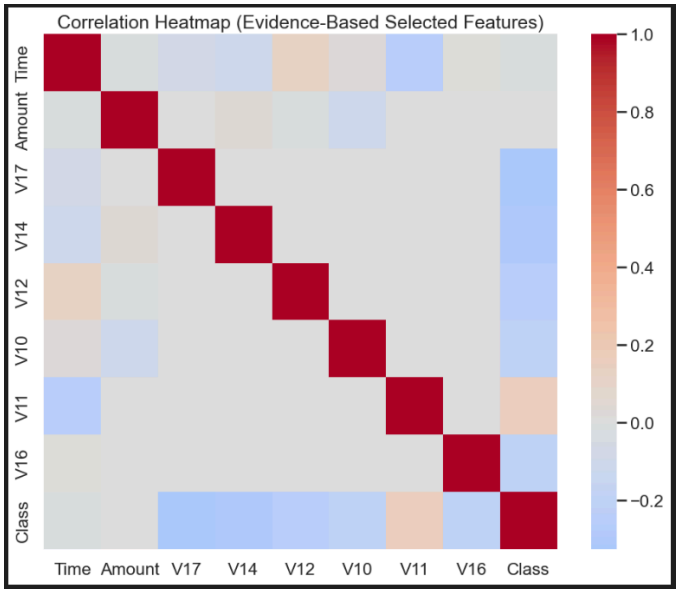
Features were selected objectively by ranking all PCA variables. Mutual information measures how much knowing a feature reduces uncertainty about whether a transaction is fraud or no fraud. Features with higher scores carry more fraud-related signals, so the top six were chosen for visualization.

Distribution comparison of selected features for fraud vs non-fraud transactions.



Feature distribution comparisons for the selected variables show visible shifts in density between the two classes. In several features, fraudulent transactions occupy different regions of the feature space, although there is still overlap. This suggests that the classes are partially separable, but not perfectly separable using a single feature, motivating the use of multi-feature machine learning models.

Correlation heatmap of selected variables including the target class



The selected-feature correlation heatmap suggests that the fraud signal is likely distributed across multiple features rather than dominated by one strongly correlated variable.

Overall, the analysis supports the use of imbalance-aware learning methods and precision-recall-focused evaluation metrics in later experiments. Despite identifying the most informative features, all 30 features were retained for model training to preserve the full information available in the dataset.

3.3 Data Splitting

To ensure fair evaluation and avoid data leakage, preprocessing was applied before model training using a consistent train, validation, and test workflow.

The dataset was first split into training and test sets using an **80-20 split**. The training portion was then split again into train and validation sets with a **75-25 split**.

This gives an effective 60-20-20 partition of:

- 170,883 training samples
- 56,962 validation samples
- 56,962 test samples.

In the improved pipelines, **stratified splitting** was used so the fraud ratio is preserved across all splits. Model selection and threshold tuning were performed on the validation set, and final performance was reported only on the held-out test set.

3.4 Data Preprocessing

For feature preprocessing, `Time` and `Amount` variables were transformed with **log1p** to reduce skewness. After that, **StandardScaler** was applied. The scaler was fit only on the training split, then the same transformation was applied to validation and test splits. This prevents leakage from validation or test data into training. Class imbalance was handled using **model-level weighting** instead of data-level resampling.

- **Neural Network models**
  - class-weighted BCEWithLogitsLoss was used through positive-class weighting.
- **XGBoost**
  - scale_pos_weight was set using the ratio of normal to fraud samples.
- **Autoencoder**
  - training was done only on normal transactions, and fraud was detected using reconstruction error with a threshold selected on validation data.

No oversampling or undersampling method was used, meaning no Synthetic Minority Over-sampling Technique (SMOTE) and no random under-sampling were applied. This avoids introducing synthetic patterns and keeps the original data distribution intact.

For training order, shuffling was enabled for training data batches, while validation and test data were not shuffled. Labels were already binary in the dataset, with Class encoded as 0 for non-fraud and 1 for fraud, and then cast to numeric tensor or array types as required by each model implementation.

3.5 Class Imbalance Handling

The dataset used in this study is highly imbalanced, with significantly more non-fraud transactions than fraud cases. This creates a challenge where models can achieve high accuracy by predicting only the majority class, while failing to detect fraud.

As a result, evaluation metrics such as **F1-score**, **precision**, **recall**, and **ROC-AUC** are more meaningful than accuracy. In particular, recall is important for identifying fraud cases, while precision ensures that false alarms are minimized.

To address this issue, techniques such as class weighting and threshold tuning are applied in later models.

4. Methodology

To address fraud detection under severe class imbalance, four models were implemented: a baseline feedforward Neural Network (NN), a weighted version of the NN, XGBoost, and an autoencoder. These models were chosen to explore different strategies for handling rare fraud cases. All models in this study use the 30 input features (Time, Amount, V1–V28) and output a binary fraud prediction.

4.1 Models Implemented

4.1.1 Baseline Feedforward Neural Network (BNN)

This implementation is what we proposed in the report. This baseline model is a simple fully connected neural network consisting of two hidden layers with **64** and **32** neurons, each followed by ReLU activation. We used **BCEWithLogitsLoss** as our loss function, and the final layer outputs a single value for binary classification.

This model serves as a reference point to evaluate how more advanced techniques affect performance. It represents a straightforward approach without any specific handling of class imbalance. A key limitation of this model is that it tends to favour the majority class, resulting in poor detection of fraud cases.

```
FraudDetectionNet(
  (fc1): Linear(in_features=30, out_features=64, bias=True)
  (bn1): BatchNorm1d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (relu1): ReLU()
  (dropout1): Dropout(p=0.5, inplace=False)
  (fc2): Linear(in_features=64, out_features=32, bias=True)
  (bn2): BatchNorm1d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (relu2): ReLU()
  (dropout2): Dropout(p=0.3, inplace=False)
  (fc3): Linear(in_features=32, out_features=1, bias=True)
)
```

4.1.2 Weighted Feedforward Neural Network (WNN)

The weighted model extends the baseline by increasing the network depth and introducing regularisation techniques. It consists of three hidden layers (**128, 64, 32 neurons**) with Batch Normalization, LeakyReLU activation, and Dropout applied between layers.

The main difference from the baseline is the use of **class weighting in the loss function**, which penalizes misclassification of fraud cases more heavily. This matters because it shifts the model's focus toward detecting the minority class, improving **recall** and **F1 score**. However, we expect that this may introduce more false positives, highlighting the trade-off between sensitivity and precision.

```
FraudDetectionNet(
  (net): Sequential(
    (0): Linear(in_features=30, out_features=128, bias=True)
    (1): BatchNorm1d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): LeakyReLU(negative_slope=0.1)
    (3): Dropout(p=0.3, inplace=False)
    (4): Linear(in_features=128, out_features=64, bias=True)
    (5): BatchNorm1d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (6): LeakyReLU(negative_slope=0.1)
    (7): Dropout(p=0.2, inplace=False)
    (8): Linear(in_features=64, out_features=32, bias=True)
    (9): LeakyReLU(negative_slope=0.1)
    (10): Dropout(p=0.1, inplace=False)
    (11): Linear(in_features=32, out_features=1, bias=True)
  )
)
```

4.1.3 XGBoost

XGBoost is a tree-based ensemble model that builds decision trees sequentially, where each new tree corrects errors made by previous ones. We trained the model using the **binary:logistic** objective, with class imbalance handled through **scale_pos_weight**. The final configuration used 300 estimators, a learning rate of 0.05, and a maximum tree depth of 5.

Unlike neural networks, XGBoost does not rely on layered representations. Instead, it captures feature interactions directly through tree splits, making it highly effective for structured tabular data. Its performance is strongly influenced by the decision threshold, which affects the balance between precision and recall.

```
XGBClassifier(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None,
              colsample_bytree=0.8, device=None, early_stopping_rounds=None,
              enable_categorical=False, eval_metric='logloss',
              feature_types=None, feature_weights=None, gamma=None,
              grow_policy=None, importance_type=None,
              interaction_constraints=None, learning_rate=0.05, max_bin=None,
              max_cat_threshold=None, max_cat_to_onehot=None,
              max_delta_step=None, max_depth=5, max_leaves=None,
              min_child_weight=None, missing=nan, monotone_constraints=None,
              multi_strategy=None, n_estimators=300, n_jobs=-1,
              num_parallel_tree=None, ...)
```

4.1.4 Autoencoder

The autoencoder is an unsupervised model designed to detect anomalies rather than perform direct classification. It consists of an encoder that compresses the input into a **16-dimensional** latent representation, and a decoder that reconstructs the original input.

We trained the model only on normal transactions, learning the underlying patterns of legitimate behaviour. During inference, transactions with high reconstruction error are treated as potential fraud. This approach differs from supervised models as it does not rely on fraud labels. Instead, it identifies deviations from normal patterns. However, it may struggle when fraudulent transactions closely resemble normal ones.

```
FraudAutoencoder(
  (encoder): Sequential(
    (0): Linear(in_features=30, out_features=64, bias=True)
    (1): BatchNorm1d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU()
    (3): Dropout(p=0.1, inplace=False)
    (4): Linear(in_features=64, out_features=32, bias=True)
    (5): ReLU()
    (6): Linear(in_features=32, out_features=16, bias=True)
    (7): ReLU()
  )
  (decoder): Sequential(
    (0): Linear(in_features=16, out_features=32, bias=True)
    (1): ReLU()
    (2): Linear(in_features=32, out_features=64, bias=True)
    (3): ReLU()
    (4): Linear(in_features=64, out_features=30, bias=True)
  )
)
```

4.2 Overall Training Setup

All models were trained using the same train, validation, and test split.

Model	**Architecture**	**Hyperparameter**
BNN	● Input layer: size 30 ● Hidden layer 1: ○ 128 neurons, followed by Batch Normalization, LeakyReLU activation, and Dropout (p = 0.3) ● Hidden layer 2: ○ 64 neurons followed by Batch Normalization, LeakyReLU activation, and Dropout (p = 0.2)	Optimizer: Adam Learning rate: 1e-3 Batch size: 2048 Epochs: 20
WNN	● Input layer: size 30 ● Hidden layer 1: ○ 128 neurons, followed by Batch Normalization, LeakyReLU activation, and Dropout (p = 0.3) ● Hidden layer 2: ○ 64 neurons, followed by Batch Normalization, LeakyReLU activation, and Dropout (p = 0.2) ● Hidden layer 3: ○ 32 neurons, followed by LeakyReLU activation and Dropout (p = 0.1)	Optimizer: Adam Learning rate: 1e-3 Batch size: 512 Epochs: 30 Early stopping on validation loss

	● Output layer: 1 neuron	
XGBoost	● Number of estimators: 300 ● Learning rate: 0.05 ● Maximum tree depth: 5 ● Column subsampling (colsample_bytree): 0.8	Model selection based on validation performance No epoch-based training
Autoencoder	Encoder: compresses the input from ● 30 → 64 → 32 → 16 dimensions ● 30 → 64 (Linear + BatchNorm + ReLU + Dropout p = 0.1) ● 64 → 32 (Linear + ReLU) ● 32 → 16 (Linear + ReLU) Decoder: reconstructs the input from ● 16 → 32 → 64 → 30 dimensions ● 16 → 32 (Linear + ReLU) ● 32 → 64 (Linear + ReLU) ● 64 → 30 (Linear)	Same optimizer and learning rate setup Trained only on non-fraud samples

4.3 Hyperparameter Tuning

We performed hyperparameter tuning for our Weighted Neural Network. Final models were selected based on validation performance, focusing on recall and PR-AUC.

5. Results

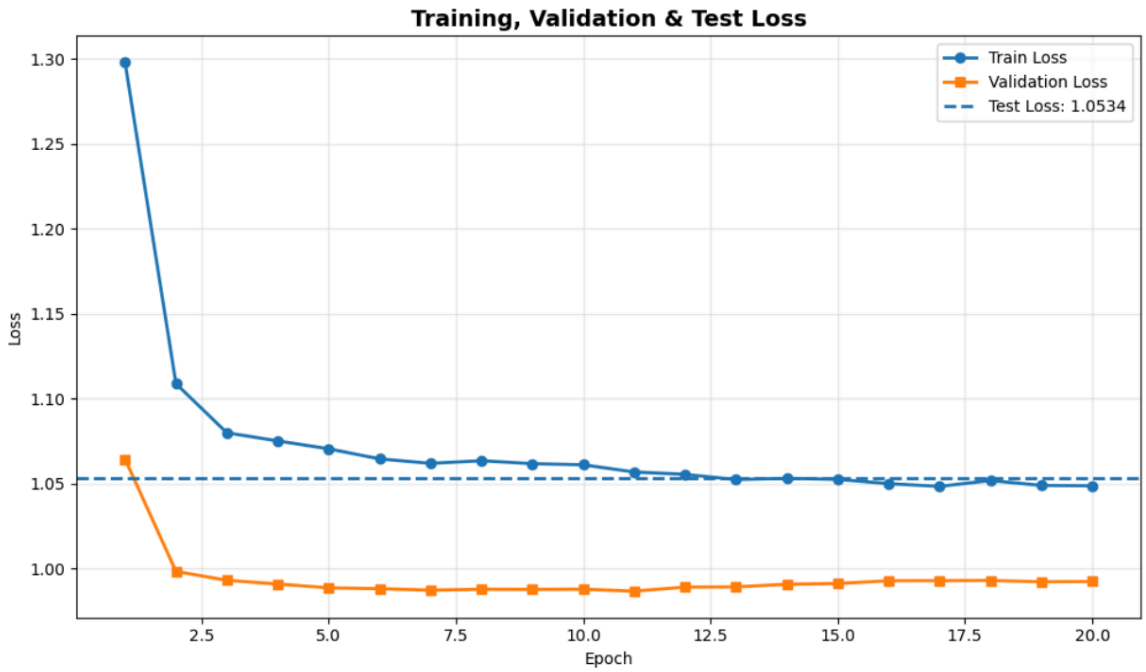
5.1 Evaluation Metrics

The following evaluation metrics were used to assess model performance:

- **Precision**: proportion of predicted fraud cases that are correct.
- **Recall**: proportion of actual fraud cases correctly identified.
- **F1-score**: harmonic mean of precision and recall.
- **ROC-AUC**: measures the model's ability to distinguish between classes.
- **PR-AUC**: summarizes the precision-recall trade-off across thresholds.

5.2 Training Curves & Evaluation Metrics

For the **BNN**, training loss decreases rapidly in the first few epochs and then stabilizes. Validation loss remains consistently lower than training loss, suggesting that the model converges quickly without clear signs of overfitting.




```
BASELINE CLASSIFIER RESULTS

Precision:   0.2788
Recall:      0.8878
F1 Score:    0.4244
ROC-AUC:     0.9757
Accuracy:    0.9959

CONFUSION MATRIX
TN:  56639   FP:    225
FN:     11   TP:     87
```

However, despite stable loss behaviour, the model exhibits poor classification balance. The model achieves a very high **recall** of approximately 89%, which means that it successfully detects most fraud cases. However, the **precision** was very low, at only 28%. This means that only roughly one quarter of the predicted frauds were actually frauds.

While this model behaviour may be acceptable in safety-critical systems where missing fraud is costly, it is not ideal to be implemented in real-world applications. Many normal users will be incorrectly flagged, and hence negatively impact user experience.

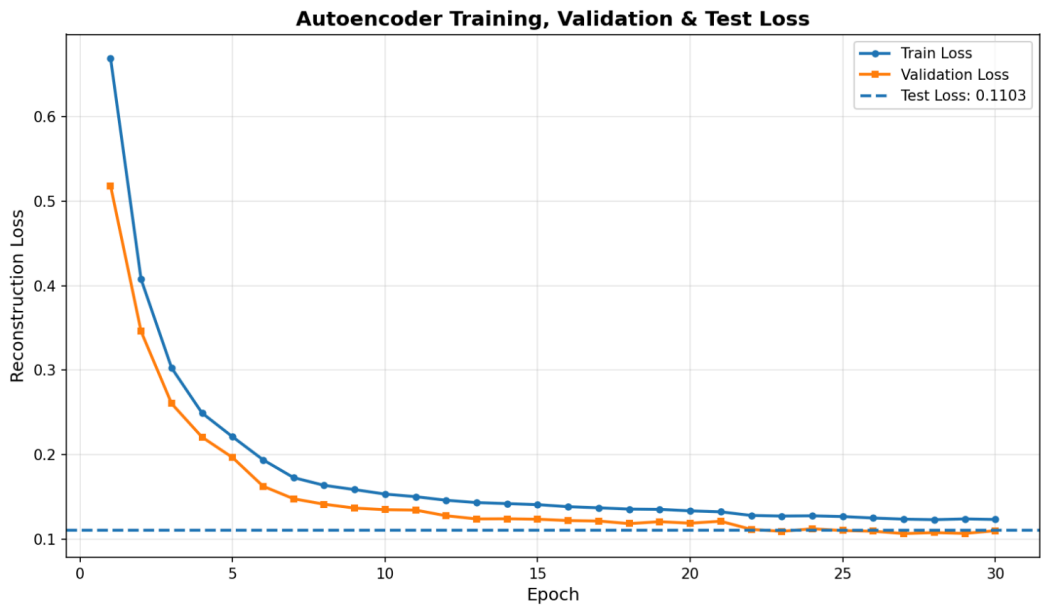
For the **WNN**, training loss decreases steadily across epochs, while validation loss initially decreases but gradually increases after several epochs. This divergence suggests mild overfitting as training progresses.

```
WEIGHTED CLASSIFIER RESULTS

Threshold: 0.95

Accuracy:  0.9994
Balanced Acc: 0.9080
Precision: 0.8333
Recall:    0.8163
F1 Score:  0.8247
ROC-AUC:   0.9818
PR-AUC:    0.7326
```

However, compared to the baseline model, the weighted network learns more meaningful patterns for the minority class, which leads to significantly improved classification performance despite slightly higher loss values.

We observe that our recall is still quite high, reaching approximately 80%, while keeping precision high at ~83% as well. This means that most of the fraud cases are actually correct. This drastically reduces false positives compared to the baseline model. This makes our weighted classifier more deployable in the real-world than our baseline.

Our **Autoencoder** shows a smooth decrease in both training and validation reconstruction loss, indicating that it successfully learns the structure of normal transactions. The gap between training and validation loss remains small, suggesting good generalization. However, since the model is trained only on normal data, low reconstruction loss does not directly translate to strong fraud detection capability.



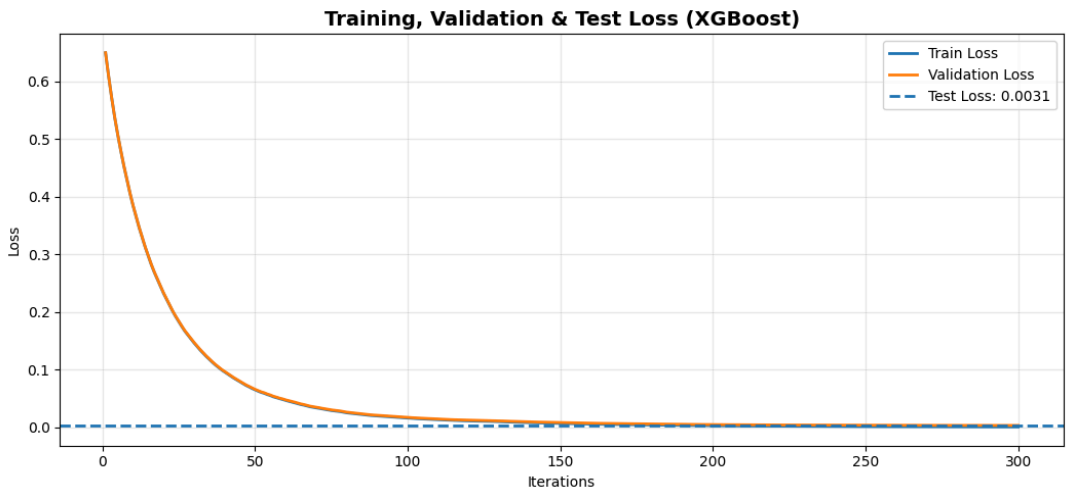

```
AUTOENCODER RESULTS

Threshold: 6.483103

Accuracy:  0.9990
Balanced:  0.7805
Precision: 0.7857
Recall:    0.5612
F1 Score:  0.6548
ROC-AUC:   0.9590
PR-AUC:    0.6526

CONFUSION MATRIX
TP:     55    FP:     15
FN:     43    TN:  56849
```

The autoencoder still achieves a high precision of ~78%, which means that most of the predicted fraud cases are actually frauds. However, the recall is significantly lower at ~56%, indicating that a substantial number of fraud cases were missed by the model.

With these results, we can observe that the autoencoder model is more conservative in predicting fraud. It only flags transactions that strongly deviate from normal patterns, which leads to significantly fewer false positives but also causes more fraud cases to be missed.

This behavior makes the model more suitable for applications where minimizing false alarms is important. However, in fraud detection scenarios where missing fraudulent transactions can be costly, the lower recall is a limitation that we have to address and would have to make up for.

For **XGBoost**, loss is tracked over boosting iterations rather than epochs. Both training and validation loss decrease rapidly and converge to very low values, indicating efficient learning and strong model fit. The close alignment between training and validation loss suggests that the model generalizes well without significant overfitting.



XGBOOST RESULTS

Threshold: 0.90

Accuracy: 0.9995
Balanced Acc: 0.9030
Precision: 0.9186
Recall: 0.8061
F1 Score: 0.8587
ROC-AUC: 0.9771
PR-AUC: 0.8697

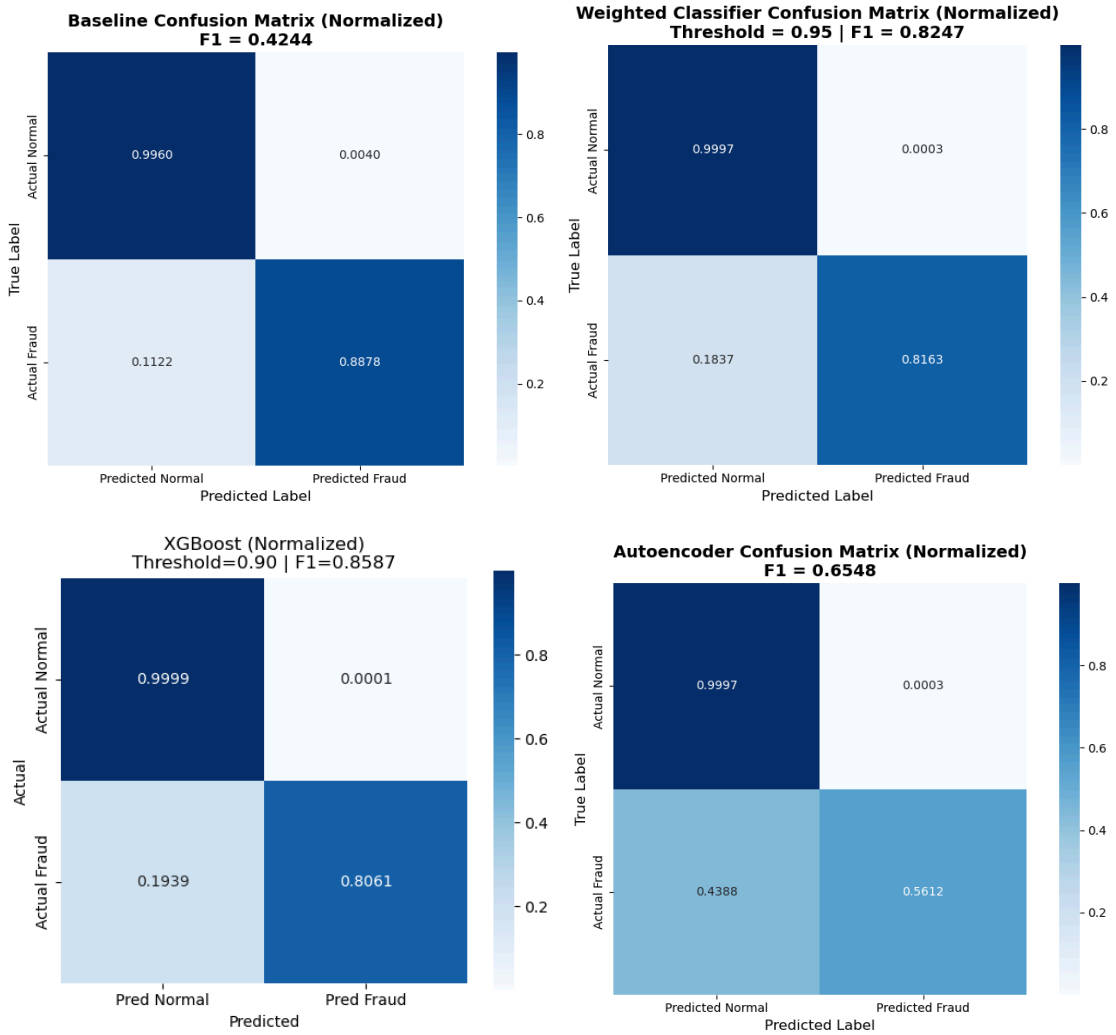
The XGBoost model outperformed the weighted neural network in terms of overall performance, achieving a higher F1 score (~86% vs ~65%) and significantly higher precision (~92% vs ~83%). This indicates that XGBoost is more effective at correctly identifying fraudulent transactions while minimizing false positives.

Although both models achieved similar recall of ~81%, the weighted classifier did so at the cost of a higher false positive rate. This trade-off is reflected in the confusion matrix (shown later), where the weighted model produced 21 false positives compared to only 5 in XGBoost.

Overall, **XGBoost demonstrates superior performance for this task** and is more suitable for real-world deployment, where minimizing false alarms while maintaining strong fraud detection capability is critical.

5.3 Final Test Performance

Confusion matrix for each model as follows:



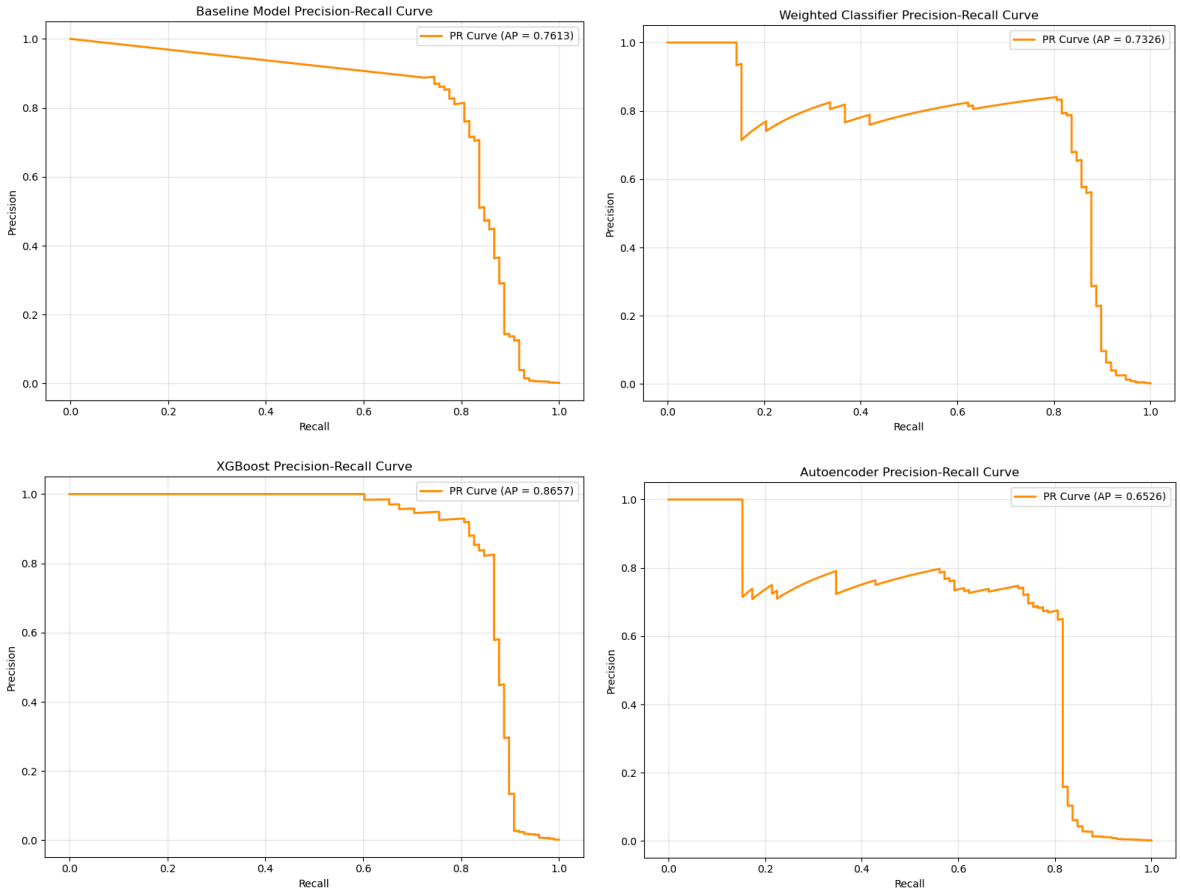
The final performance of all models on the test set is summarized below:

Model	Precision	Recall	F1-score	ROC-AUC	PR-AUC
BNN	0.2788	**0.8878**	0.4244	0.9757	0.9959
WNN	0.8333	0.8163	0.8247	**0.9818**	0.7326
XGBoost	**0.9186**	0.8061	**0.8587**	0.9790	**0.8657**
Autoencoder	0.7857	0.5612	0.6548	0.9590	0.6526

Overall, these results point to a few key takeaways. First, handling class imbalance is clearly important, as seen from the large improvement between the **BNN** and the **WNN**. Second, supervised models perform better than the unsupervised **Autoencoder** in this setting, suggesting that having labeled data makes a real difference for fraud detection. Finally, **XGBoost** achieves the best overall balance, making it a strong choice for highly imbalanced tabular fraud detection tasks.

5.4 Threshold Analysis

To further understand model behaviour, precision-recall curves were used to examine the trade-off between detecting fraud and limiting false alarms.



As shown, all models experience the usual threshold trade-off: when the decision threshold is lowered, recall increases because more fraud cases are detected, but precision decreases because more normal transactions are incorrectly flagged as fraud. Conversely, when the threshold is raised, precision improves but recall falls, meaning more fraud cases are missed.

Among the models, **XGBoost shows the strongest precision-recall trade-off**, with the highest PR-AUC of **0.8657**. Its curve remains close to the top-right region for a large portion of the range, indicating that it maintains high precision even as recall increases. This suggests that XGBoost provides the most robust performance across different threshold choices.

WNN also demonstrates a strong trade-off, with a PR-AUC of **0.7326**. Its curve is generally stable, although it drops more sharply than XGBoost at higher recall values. This indicates that while class weighting greatly improves fraud detection compared to the baseline model, its performance is somewhat more sensitive to threshold selection.

BNN achieves a PR curve area of approximately **0.7613**. Although it attains high recall, the curve falls more sharply in the high-recall region, showing that this comes at the expense of precision. This is consistent with its low precision score in the final test results, where many normal transactions were incorrectly classified as fraud.

The **Autoencoder** has the weakest precision-recall trade-off, with a PR-AUC of approximately **0.6526**. Its curve drops earlier and more noticeably than the supervised models, indicating less reliable separation between fraudulent and non-fraudulent transactions. This suggests that reconstruction-error-based anomaly detection is less effective than supervised learning for this dataset.

Overall, the precision-recall curves reinforce the conclusions from the final test metrics: **XGBoost** provides the best overall balance and the **WNN** is a strong improvement over the baseline, and the autoencoder is less competitive in this setting.

6. Model Limitations

Although we managed to receive pretty promising results, there are still limitations that we have to take into account since we are training from scratch. Some of these limitations are listed below.

6.1 Incorrect Model Implementation

In our proposal, we proposed using a Sigmoid activation in the output layer together with BCEWithLogitsLoss. We eventually realized this combination is incorrect because BCEWithLogitsLoss already includes a Sigmoid activation.

Since our first try followed our proposal, the model essentially applied Sigmoid twice during training. This weakened learning and reduced training stability, leading to a lot of false negatives. We used this original, uncorrected implementation as our baseline in the evaluation, since it reflects the model exactly as proposed. To fix this, we removed Sigmoid from the model output and kept BCEWithLogitsLoss with class weighting. Sigmoid is now applied only during evaluation to convert logits into probabilities.

6.2 Model-Specific Failure Patterns and Malfunction Examples

Each model demonstrates different types of weaknesses, which can be observed through specific malfunction patterns such as false positives, false negatives, and threshold-sensitive misclassifications.

The **Baseline Neural Network** struggles with class imbalance, leading to excessive false positives. This is seen from its very low precision (~28%) despite high recall (~89%), meaning that many normal transactions were incorrectly flagged as fraud. This happens because the model does not learn a strong enough separation between fraud and non-fraud cases.

The **Weighted Neural Network** improves minority class detection but shows mild overfitting, as observed from divergence between training and validation loss. Its malfunction is less severe, but performance remains sensitive to threshold selection, where small changes can noticeably shift the balance between false positives and false negatives.

The **Autoencoder** tends to miss fraud cases that resemble normal transactions. This can be seen in its lower recall (~56%), showing that a substantial portion of fraud cases were not detected. The reason is that fraud is identified only through reconstruction error, and some fraudulent transactions are reconstructed too well to be treated as anomalies.

Although **XGBoost** performs best overall, it remains sensitive to the choice of threshold. Some borderline legitimate transactions may be flagged as fraud, while subtle fraud cases may be missed. This happens because its final predictions still depend on the selected classification threshold and the precision-recall trade-off.

6.3 Feature Limitations

The dataset consists of **PCA-transformed features (V1–V28)**, which introduces several limitations. As mentioned earlier, the features are anonymized and lack interpretability, and important domain-specific information (such as transaction location or merchant type) is not available. As a result, meaningful real-world patterns may be compressed or distorted during the transformation process. Consequently, models may struggle to fully capture the underlying characteristics of fraudulent behavior. In addition, this lack of interpretability makes it difficult to determine which features meaningfully contribute to identifying a transaction as fraudulent.

6.4 Model Architecture Limitations

The feedforward neural networks used in this study are relatively simple and may not fully capture complex interactions between features. While effective for baseline comparisons, their representational capacity is limited compared to more advanced architectures.

In addition, no temporal or sequential information is incorporated, even though fraudulent behaviour often exhibits time-dependent patterns (e.g., unusual transaction sequences or bursts of activity). Ignoring this aspect may limit the model's ability to detect certain types of fraud.

The autoencoder also presents inherent limitations, as it relies solely on reconstruction error to identify anomalies. However, a high reconstruction error does not always correspond to fraudulent behaviour, which can lead to misclassification of both normal and fraudulent transactions.

6.5 Threshold Sensitivity

All models demonstrate sensitivity to the choice of classification threshold. Lower thresholds tend to increase recall by capturing more fraudulent transactions, but this comes at the cost of reduced precision and a higher number of false positives. Conversely, higher thresholds improve precision but increase the likelihood of missing fraudulent cases (false negatives).

As such, selecting an appropriate threshold is critical and should be guided by the specific business context and risk tolerance. In practice, this involves balancing the cost of missed fraud against the impact of incorrectly flagging legitimate transactions.

7. Potential Improvements

Several improvements can be made to address the limitations identified. First, more advanced techniques for handling imbalance can be explored, such as SMOTE or hybrid resampling methods, to provide the model with more representative fraud samples during training.

Incorporating temporal features or using sequence-based models such as LSTMs may also help to capture behavioural patterns across transactions, which are often indicative of fraud. Threshold optimization can also be further refined using validation data to better balance precision and recall based on application requirements. For the autoencoder, performance could be improved by experimenting with different architectures or combining it with supervised models in a hybrid approach, allowing both anomaly detection and classification signals to be used.

Finally, additional feature engineering, including interaction features or domain-specific indicators, may help improve model performance by providing more meaningful input representations.

8. Discussion

In this project, we first learned that handling class imbalance is crucial for anomaly detection. The large improvement from the baseline neural network to the weighted neural network

highlights how class weighting significantly improves performance, particularly in balancing precision and recall.

Second, although we learned that neural networks are pretty capable of learning complex patterns especially for anomaly detection, it would be more reliable to deploy a state-of-the-art model such as XGBoost. From our experimentations, we observed that XGBoost outperformed our deep learning model.

Third, there is a clear trade-off between recall and precision. The baseline model achieves high recall but suffers from extremely low precision, making it impractical. In contrast, XGBoost and the weighted neural network achieve a more balanced trade-off, which is more suitable for real-world deployment.

From a practical perspective, fraud detection systems must consider the cost of errors:

- High false positives lead to poor customer experience.
- High false negatives lead to financial loss.

The results suggest that XGBoost provides the most practical balance, while the weighted neural network offers a strong alternative when deep learning is preferred. Finally, the findings show that supervised learning is more effective than unsupervised approaches in this setting, as the autoencoder struggles to reliably detect fraud without labeled examples.

9. Reproducibility / How to Run

The experiments were implemented using **Python and PyTorch**, with additional libraries including NumPy, Pandas, Scikit-learn, and XGBoost.

9.1 Environment

- Python version: 3.x
- Framework: PyTorch
- Key libraries: NumPy, Pandas, Scikit-learn, XGBoost, Matplotlib

9.2 Dataset

- Source: Kaggle Credit Card Fraud Detection dataset
- Link: https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud

9.3 GitHub Repository

- Link: https://github.com/xenaria/credit-card-fraud-detection.git
    - Notebook Reading and Execution Order

        i. BaselineFFN_original
        ii. BaselineFFN_corrected
        iii. WeightedClassifier
        iv. Autoencoder
        v. SOTA_XGBoost

9.3.1 How to navigate

1. The 4 main notebooks for our model implementation were:
    - BaselineFFN_original.ipynb (The model proposed in our Project Proposal)
    - WeightedClassifier.ipynb
    - Autoencoder.ipynb
    - SOTA_XGBoost.ipynb
2. Other notebooks
    - BaselineFFN_corrected.ipynb
        i. Similar to BaselineFFN_original.ipynb one but without sigmoid layer at the end, outputting raw logit directly. Essentially a more regularized model.
    - WeightedClassifier_ft.ipynb
        i. Separate notebook for choosing and fine tuning hyperparameters for our weighted classifier.
        ii. Saving model to **.pt** is also in this notebook.
3. Model output
    - Best_weighted_fraud.pt

9.4 Steps to Run

1. **Data Preprocessing**
    - Apply log transformation to Time and Amount
    - Standardize features using training set statistics
    - Split data into train, validation, and test sets (60-20-20)
2. **Model Training**
    - Train models using specified configurations
    - Apply class weighting for neural networks and XGBoost
    - Use early stopping based on validation performance
3. **Model Evaluation**
    - Evaluate using precision, recall, F1-score, ROC-AUC, and PR-AUC
    - Perform threshold tuning on validation set
4. **Saving and Loading Models**
    - Models are saved as **.pt** files (PyTorch)
    - Load using **model.load_state_dict()** for inference
5. **Reproducing Results**
    - Run training cells in sequence
    - Generate training curves and evaluation metrics
    - Use saved models to reproduce final results

10. Team Contributions

The tasks were distributed evenly for this project. Common contributions include hyperparameter tuning, checking, trying, and validating each other's models, as well as improving the pipeline and writing the report. More specific task allocations are listed below.

Name	Contributions
Pagdilao Geoffrey Cyd Caraig	● Exploratory Data Analysis ● Basic Neural Network model implementation & training pipeline
Karen Neo	● Weighted Neural Network design and implementation ● Autoencoder model implementation
Lindero Dianthe Marithe Lumagui	● XGBoost model implementation ● Evaluation metrics, visualization, and result interpretation

11. Conclusion

In this project, multiple machine learning approaches were explored for credit card fraud detection under severe class imbalance.

Among the models evaluated, XGBoost achieved the best overall performance, with the highest F1-score and PR-AUC, and the most balanced trade-off between precision and recall. The results highlight three key points: handling class imbalance is essential, model choice significantly affects performance, and supervised methods outperform unsupervised approaches in this setting.

Overall, the findings show that effective fraud detection is not just about model accuracy, but about managing the trade-offs between different types of errors in a practical context.